COMP 182 / LECTURE 04

Abstract data types and array lists

Contracts, collections, and a Java implementation

What you will be able to do

Specify a list through its operations and contracts.

Use collections, iterators, and forEach.

Trace insertion, deletion, and array growth.

Explain the cost of each list operation.

Data structures and ADTs

Data structure: a concrete way to organize stored data.

ADT: values, operations, and their specified behavior.

List ADT: an ordered sequence with indexed operations.

Abstraction and information hiding

Procedural abstraction describes what a method does.

Data abstraction describes how a collection behaves.

Private fields keep storage behind the public operations.

Discussion 1: changing the storage

A registration system uses a list of student names.

Replace an array with linked nodes.

What should stay the same? What might change?

A useful operation contract

get(index)

Requires: 0 <= index < size

Returns: the element at index

Invalid index: throws IndexOutOfBoundsException

Effect: the list is unchanged

Quiz 1: contract or implementation?

Classify each statement.

A. get(0) returns the first element.

B. Elements are stored in Object[].

C. Insertion shifts elements to the right.

D. An invalid get index throws an exception.

The collection family

Iterable supplies iterator().

Collection extends Iterable.

List, Set, and Queue describe different collection behaviors.

Map associates keys with values and is separate from Collection.

Java Collection API

Collection operations

Inspect: size(), isEmpty(), contains(value)

Change: add(value), remove(value), clear()

Combine: addAll(c), removeAll(c), retainAll(c)

Export: toArray()

Open Java: CollectionDemo.java

Quiz 2: collection output

```
Collection<String> names = new ArrayList<>(
    Arrays.asList("Ada", "Lin", "Ada"));
names.remove("Ada");
System.out.println(names.size());
System.out.println(names.contains("Ada"));
```

Open Java: CollectionDemo.java

The List contract

A list is an ordered sequence with indices starting at 0.

get(i) retrieves and set(i, e) replaces.

add(i, e) inserts and remove(i) deletes.

indexOf(e) and lastIndexOf(e) search for a position.

Index bounds

Operation	Valid index
get, set, remove	0 <= i < size
add at a position	0 <= i <= size

An empty list allows add(0, e), but not get(0).

Open Java: MyArrayList.java

Quiz 3: two remove methods

```
List<Integer> values = new ArrayList<>(
    Arrays.asList(10, 20, 10));
System.out.println(values.remove(1));
System.out.println(values.remove(Integer.valueOf(10)));
System.out.println(values);
```

Open Java: ListPitfallsDemo.java

Iterator traversal

```
Iterator<String> it = names.iterator();
while (it.hasNext()) {
    String name = it.next();
    System.out.println(name);
}
```

Open Java: IterationDemo.java

Safe removal during traversal

```
Iterator<Integer> it = values.iterator();
while (it.hasNext()) {
    if (it.next() % 2 == 0) {
        it.remove();
    }
}
```

Open Java: IterationDemo.java

Enhanced for and forEach

```
for (String name : names) {
    System.out.println(name);
}

names.forEach(System.out::println);
```

Open Java: IterationDemo.java

Discussion 2: choose a traversal

You need to remove every even integer.

A. Enhanced for plus values.remove(value)

B. Explicit iterator plus it.remove()

Which is appropriate here, and why?

One list contract, different storage

Array-backed list: an array plus a logical size.

Linked list: nodes that reference other nodes.

Clients call list operations in either implementation.

Storage choices affect time and memory costs.

Open Java: ContractDemo.java

A small teaching interface

```
interface MyList<E> extends Iterable<E> {
    int size();
    E get(int index);
    void add(int index, E value);
    E remove(int index);
    // More operations in MyList.java
}
```

Open Java: MyList.java

Size and capacity

size = number of logical elements
capacity = length of the backing array
Invariant: 0 <= size <= capacity

Index	**0**	**1**	**2**	**3**
Storage	A	B	C	null
In list?	yes	yes	yes	no

Only indices 0 through size - 1 are in the list.

Open Java: MyArrayList.java

Retrieval and replacement

```
public E set(int index, E value) {
    E old = get(index);
    data[index] = value;
    return old;
}
```

Open Java: MyArrayList.java

Insertion at index 1

Insert X into [A, B, C]. Shift from right to left.

State	0	1	2	3
Before	A	B	C	null
Shifted	A	B	B	C
After	A	X	B	C

Two existing elements move. Capacity stays 4.

Open Java: ArrayListDemo.java

The insertion loop

```
ensureCapacity(size + 1);
for (int j = size; j > index; j--) {
    data[j] = data[j - 1];
}
data[index] = value;
size++;
```

Open Java: MyArrayList.java

Deletion at index 1

Remove B from [A, B, C, D]. Shift from left to right.

State	0	1	2	3
Before	A	B	C	D
Shifted	A	C	D	D
After	A	C	D	null

After removal: size 3, capacity 4

Open Java: MyArrayList.java

Quiz 4: trace the array list

Start with [A, B, C], size 3, capacity 4.

1. add(1, "X")

2. remove(2)

3. add("D")

4. add("E")

Give the list, size, and capacity after each step.

Open Java: ArrayListDemo.java

Growing the backing array

When size equals capacity, allocate a larger array.

Copy the existing elements and switch the reference.

Our teaching implementation doubles capacity.

The JDK does not promise a specific growth multiplier.

Open Java: MyArrayList.java

Costs of array-backed operations

Let n be the current size and i a valid index.

Operation	Time
get(i), set(i, e)	O(1)
append	O(1) amortized, O(n) worst case
insert at i	O(n) worst case
remove at i	O(n - i), with at least O(1) work
indexOf, lastIndexOf	O(n) worst case

Cost depends on both shifting and occasional growth.

Discussion 3: choose an implementation

A. Read randomly selected elements by index.

B. Insert repeatedly at the front.

C. Append many items, then read them.

Which storage would you consider for each workload?

Lab: work through the interface

Complete ListLab.java.

1. Count matches using null-safe equality.

2. Remove all matches without skipping adjacent matches.

Test [A, A, null, B, A] with target A.

Open Java: ListLab.java

Boundary cases and checks

An empty list can accept add(0, e).

get(size) and remove(size) are invalid.

Removing the last item must leave a usable empty list.

Trimming to zero must still permit the next append.

Open Java: ListChecks.java

Exit quiz

1. Why is size different from capacity?

2. Which direction does insertion shift? Why?

3. Can get(size) ever be valid?

4. Why is append amortized O(1), but sometimes O(n)?

Review and Java examples

An ADT defines behavior through a contract.

Iteration hides the storage used by a collection.

Array lists combine direct access with shifting and growth.

Use the linked source files to test your predictions.

Open the example guide